

Chapter 9: Population Methods

Algorithms for Optimization, 2nd Edition (2026)

Kochenderfer & Wheeler

Adapted for Lecture Slides

2026

Chapter Outline

- 1 Population Iteration
- 2 Genetic Algorithms
- 3 Differential Evolution
- 4 Particle Swarm Optimization
- 5 Firefly Algorithm
- 6 Cuckoo Search
- 7 Hybrid Methods
- 8 Summary & Exercises

Motivation: Why Use a Population of Solutions? I

All the optimization methods covered in previous chapters share one structural feature: they maintain a *single* candidate design point $\mathbf{x}$ and iteratively improve it. This single-point strategy works well for smooth, unimodal objectives, but it has a fundamental weakness: once the search converges to a local minimum, there is no information elsewhere in the design space to draw on.

Population methods take a different approach. Instead of tracking one design, they maintain a *collection* (a “population”) of m (m) candidate solutions $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(m)}$ and evolve all of them simultaneously. Here, $\mathbf{x}^{(i)}$ ($\mathbf{x}$ superscript i) denotes the i -th (i -th) individual in the population.

Four Key Advantages of Population Methods

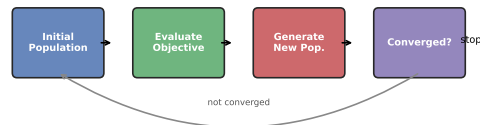
1. Information sharing across the design space.

Because different individuals occupy different regions, the algorithm can detect structure (ridges, valleys) that a single-point method would never see.

2. Better ability to escape local minima. An individual trapped near a local minimum can be “rescued” by genetic material from a better-located individual.

3. Natural parallelism. All m (m) function evaluations in a generation are independent and can be computed

Population Iteration Loop



The fundamental population loop: initialise m individuals, evaluate the objective f at each, generate a new population using a method-specific rule, and check for convergence. This loop structure is shared by all methods in this chapter.

Population Iteration: The General Template I

Before studying specific algorithms (genetic algorithms, differential evolution, PSO, etc.), it is helpful to see that they all share the same high-level skeleton.

General Population Method — Algorithm 9.1

- ➊ **Initialise** the population $\mathcal{P} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$, typically by drawing each individual uniformly at random from the feasible design space.
- ➋ For $k = 1, 2, \dots, k_{\max}$ (k sub max, the maximum number of generations):
 - Ⓐ Evaluate $f(\mathbf{x}^{(i)})$ for every individual i in the current population.
 - Ⓑ **Generate** a new population using a method-specific rule that typically rewards lower-objective individuals (selection), combines information from multiple parents (crossover), and introduces randomness (mutation).
- ➌ Return the best individual found: $\mathbf{x}^* = \arg \min_i f(\mathbf{x}^{(i)})$.

Here $k_{\max}$ (k sub max) is a user-specified budget on the number of generations, and $\arg \min$ (“argmin”, meaning “the argument that minimises”) extracts the individual with the smallest objective value at the end.

Population Iteration: The General Template II

Key Insight

The only part that differs between genetic algorithms, differential evolution, PSO, and other methods in this chapter is *step 2(b)*: how the new population is generated from the current one. The initialisation and termination logic are identical.

Population Iteration: Python Skeleton

```
1 import numpy as np
2
3 def population_method(f, bounds, pop_size=50, k_max=100, rng=None):
4     """Generic population loop. Override 'step' to specialise."""
5     rng = rng or np.random.default_rng()
6     lo, hi = np.array(bounds).T           # lower and upper bounds per dimension
7     # Step 1: uniform initialisation
8     population = rng.uniform(lo, hi, (pop_size, len(lo)))
9     for _ in range(k_max):
10         # Step 2a: evaluate objective at every individual
11         y = np.array([f(x) for x in population])
12         # Step 2b: method-specific update (returns new population)
13         population = step(population, y, rng)
14     # Return best individual
15     y = np.array([f(x) for x in population])
16     return population[np.argmin(y)]
```

Listing 1: Generic population method skeleton (Python / NumPy)

In this skeleton, `bounds` is a list of `(lo, hi)` pairs, one per dimension. The array `population` has shape `(pop_size, n)` where `n` is the number of design variables. The function `step` — which you override when implementing a specific method — takes the current population and its objective values and returns a new population of the same size.

Genetic Algorithms: Biological Inspiration and Overview I

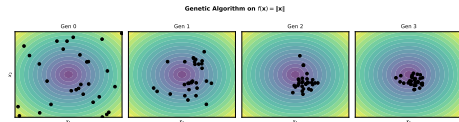
Genetic Algorithms (GAs) are the oldest and most well-known class of population methods. They were developed by John Holland in the 1970s and draw their inspiration from Darwinian natural selection: individuals that are better adapted to their environment survive and reproduce more often, gradually shifting the population toward higher fitness.

In the optimization context, “fitness” corresponds to having a low objective value $f(\mathbf{x})$. Each individual $\mathbf{x}$ is called a *chromosome*, and its components are called *genes*.

Three Core Operators — Algorithm 9.2

A GA applies three operators each generation:

- 1 **Selection** — choose which individuals get to “reproduce” (become parents of the next generation), favouring those with lower objective values.
- 2 **Crossover** — combine the chromosomes (design vectors) of two parents to create offspring. This is the mechanism by which good building blocks discovered in different parts of the population get combined.
- 3 **Mutation** — randomly perturb individual genes of the



Four successive generations of a genetic algorithm minimising $f(\mathbf{x}) = \|\mathbf{x}\|$ (the Euclidean distance from the origin). The population visibly concentrates toward $\mathbf{x} = \mathbf{0}$ (the global minimum) as generations pass.

Genetic Algorithms: Biological Inspiration and Overview II

Notation: The Fitness Formula

In the formula above, $\max\{f(\mathbf{x}^{(1)}), \dots, f(\mathbf{x}^{(m)})\}$ is the largest (worst) objective value in the current population. Subtracting $f(\mathbf{x}^{(i)})$ from this maximum gives a non-negative number: the worse individual has fitness zero, while the best individual has the highest fitness. This ensures that fitness values are always ≥ 0 (greater than or equal to zero), which is required by roulette-wheel selection (discussed next).

Key Insight: Why This Fitness Formula Works

The fitness formula flips the objective: a *lower* $f(\mathbf{x}^{(i)})$ gives a *higher* fitness_{*i*}. This means that the best individual in the population always has the largest fitness score, and the worst individual always has fitness exactly zero (making it the least likely to be selected as a parent). As the population improves over generations, the scale of fitness scores shrinks — a natural self-calibration effect.

9.2.1 Selection: Choosing Which Individuals Reproduce I

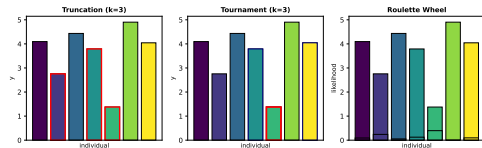
Selection is the mechanism by which the genetic algorithm applies “survival of the fittest”. The goal is to choose parent pairs for the next generation so that individuals with lower objective values (higher fitness) are more likely to pass their genes on. Three common selection methods are described below.

Method 1: Truncation Selection

Keep only the k (k) best individuals (those with the lowest objective values) and discard all others. New parent pairs are sampled uniformly at random from this elite subset of size k . This method is conceptually the simplest: you just sort the population, take the top k , and sample from those. The trade-off is that it can reduce genetic diversity rapidly, because only k unique parents are available.

Method 2: Tournament Selection

To choose each parent, draw k (k) individuals at random from the full population and select the one with the best objective value among those k competitors. The “tournament size” k controls how strongly selection favours



Left: Truncation selection — the three individuals with red borders are the top- k elite; only these are used as parents. **Centre:** Tournament selection — a random subset of individuals competes and the winner (circled) becomes a parent. **Right:** Roulette wheel — the bar height for each individual represents its selection probability; the coloured band is proportional to fitness.

9.2.1 Selection: Choosing Which Individuals Reproduce II

Key Insight: Which Selection Method to Use?

Truncation is simple and fast but can reduce diversity; it works well when the objective landscape is relatively smooth. Tournament selection is the most flexible: adjusting k lets you interpolate between random selection ($k = 1$) and pure elitism ($k = m$). Roulette wheel can be unstable when one individual is far better than all others (it dominates the selection probability), but it is the most theoretically natural formulation. For most practical problems, tournament or truncation selection is recommended.

Selection: Python Implementation

```
1 import numpy as np
2
3 def truncation_select(y, k, rng):
4     """Return parent-index pairs from the top-k lowest-objective individuals."""
5     parents = np.argsort(y)[:k]           # indices of k smallest objective values
6     pairs = rng.choice(parents, (len(y), 2))
7     return pairs
8
9 def tournament_select(y, k, rng):
10    """Tournament selection: k random competitors per parent slot."""
11    def pick_one():
12        contenders = rng.choice(len(y), k, replace=False) # k random competitors
13        return contenders[np.argmin(y[contenders])]        # winner = lowest y
14    return [(pick_one(), pick_one()) for _ in range(len(y))]
15
16 def roulette_select(y, rng):
17    """Fitness-proportional (roulette-wheel) selection."""
18    fitness = np.max(y) - y                # invert: lower y => higher fitness
19    fitness = np.maximum(fitness, 0)        # guard against numerical negatives
20    if fitness.sum() == 0:
21        fitness = np.ones_like(fitness)    # fallback: all equal if all identical
22    probs = fitness / fitness.sum()         # normalise to probabilities
23    idx = rng.choice(len(y), (len(y), 2), p=probs)
24    return idx
```

Listing 2: Three selection methods (Python / NumPy)

9.2.2 Crossover: Combining Parent Chromosomes I

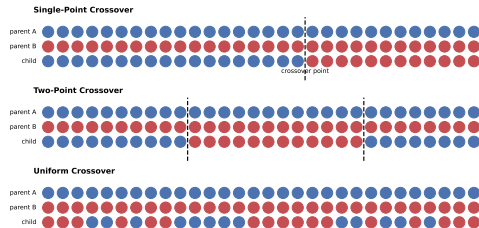
Crossover (also called recombination) is how a genetic algorithm combines information from two parent individuals to create an offspring. The idea is that if parent A is good at one aspect of the problem and parent B is good at another, their offspring might inherit both strengths. Below are four crossover schemes, ranging from simple discrete methods to a continuous interpolation approach.

Scheme 1: Single-Point Crossover

A single crossover point i is drawn uniformly from $\{1, \dots, n-1\}$ (1 to n minus 1), where n (n) is the chromosome length (number of design variables). The offspring inherits genes 1 through i from parent A ($\mathbf{x}_a$) and genes $i+1$ through n from parent B ($\mathbf{x}_b$). This preserves long contiguous “building blocks” from each parent.

Scheme 2: Two-Point Crossover

Two crossover points $i < j$ (i less than j) are drawn uniformly. The offspring inherits genes 1: i from parent A, genes $i+1$: j from parent B, and genes $j+1$: n from parent A.



Blue segments are genes inherited from parent A; red segments are genes inherited from parent B. **Top:** single-point crossover — one split divides the chromosome into two segments. **Centre:** two-point crossover — a contiguous segment from parent B is embedded inside parent A's chromosome. **Bottom:** uniform crossover — each gene is independently and randomly assigned to one of the two parents.

9.2.2 Crossover: Combining Parent Chromosomes II

Key Insight: Why Crossover Matters

Crossover is the mechanism that makes genetic algorithms qualitatively different from simple parallel random restarts. If parent A has found good values for variables 1–5 and parent B has found good values for variables 6–10, a single-point crossover at position 5 produces an offspring that may be good at *all* variables at once. This “building block” combination is the core theoretical justification for why GAs work.

Crossover: Python Implementation

```
1 import numpy as np
2
3 def single_point_crossover(a, b, rng):
4     """Single-point crossover of two real-valued vectors."""
5     i = rng.integers(1, len(a))          # crossover point in {1, ..., n-1}
6     return np.concatenate([a[:i], b[i:]] # first i genes from a, rest from b
7
8 def two_point_crossover(a, b, rng):
9     n = len(a)
10    i, j = sorted(rng.choice(n, 2, replace=False)) # two distinct cut points
11    return np.concatenate([a[:i], b[i:j], a[j:]] # a, then b-segment, then a
12
13 def uniform_crossover(a, b, p, rng):
14     """Each gene from parent a with probability p, else from parent b."""
15     mask = rng.random(len(a)) < p          # True where we take from a
16     return np.where(mask, a, b)
17
18 def interpolation_crossover(a, b, lam=0.5):
19     """Convex combination: offspring = (1-lambda)*a + lambda*b."""
20     return (1 - lam) * a + lam * b
```

Listing 3: Four crossover operators (Python / NumPy)

Each function takes two parent arrays a and b of equal length n and returns a single offspring array of the same length. In `two_point_crossover`, `sorted()` guarantees $i < j$. In `uniform_crossover`, `mask` is a boolean array: True positions take from a , False positions take from b .

9.2.3 Mutation: Introducing New Genetic Material I

Mutation is the third and final operator in a genetic algorithm. Even with selection and crossover operating perfectly, a GA can only explore combinations of gene values that were already present in the initial population. If the optimal solution requires a gene value that no individual in the initial population possesses, crossover alone can never produce it. Mutation solves this problem by randomly perturbing individual genes, ensuring that new gene values can always be discovered.

9.2.3 Mutation: Introducing New Genetic Material II

[\[fragile\]](#) Genetic Algorithm: Complete Python Exa

Distribution Mutation — Algorithm 9.5

Each gene v (v) in the offspring chromosome is mutated independently with probability λ (lambda, the mutation rate). When a gene is selected for mutation, a random perturbation ε (epsilon) drawn from a distribution $\mathcal{D}$ is added to it:

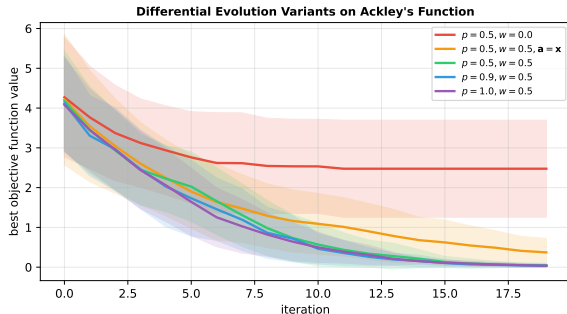
9.2.3 Mutation: Introducing New Genetic Material III

The term $w(\mathbf{b} - \mathbf{c})$ is the “difference vector”. Its magnitude automatically scales with the current diversity of the population: when the population is spread out (early in the search), $\|\mathbf{b} - \mathbf{c}\|$ is large and perturbations are large, enabling broad exploration. As the population converges, $\|\mathbf{b} - \mathbf{c}\|$ shrinks naturally, and perturbations become small, enabling fine-tuning. This self-adaptation of step size is a key strength of DE over methods that use a fixed σ (sigma) throughout.

Differential Evolution: Python Implementation

```
1 import numpy as np
2
3 def differential_evolution(f, bounds, pop_size=20, k_max=200,
4                           p=0.5, w=0.5, rng=None):
5     rng = rng or np.random.default_rng()
6     lo, hi = np.array(bounds).T
7     n = len(lo)
8     pop = rng.uniform(lo, hi, (pop_size, n))    # uniform initialisation
9
10    for _ in range(k_max):
11        new_pop = pop.copy()
12        for i in range(pop_size):
13            # Step 1: pick 3 distinct individuals != i
14            choices = [j for j in range(pop_size) if j != i]
15            a, b, c = pop[rng.choice(choices, 3, replace=False)]
16            # Step 2: form trial vector  $z = a + w*(b - c)$ 
17            interim = a + w * (b - c)
18            # Step 3: binomial crossover with guaranteed one component
19            mask = rng.random(n) < p
20            if not mask.any():
21                mask[rng.integers(n)] = True    # force at least one change
22            child = np.where(mask, interim, pop[i])
23            child = np.clip(child, lo, hi)
24            # Step 4: greedy acceptance
25            if f(child) < f(pop[i]):
26                new_pop[i] = child
27    pop = new_pop
```

Differential Evolution: Parameter Analysis and Variants I



Five DE variants tested on Ackley's function (1000 independent trials, population size $m = 20$, 2D problem). Each curve shows the mean best objective value ± 1 standard deviation over iterations. Lower is better.

Key Findings (Exercise 9.4)

The five variants differ in how the parameters p and w are set, or how the anchor point $\mathbf{a}$ is chosen.

Case 1: $w = 0$ (no differential perturbation).

When $w = 0$, the trial vector simplifies to $\mathbf{z} = \mathbf{a}$. No new gene values outside the current population range can ever be generated. The algorithm stagnates immediately — this case performs worst.

Case 2: $\mathbf{a} = \mathbf{x}^{(i)}$ (perturb around self). Using the current individual as the anchor biases the search toward local exploration around each individual's own position. This variant is harder to escape local minima.

Case 3: Standard $p = 0.5, w = 0.5$. The default configuration balances global and local search. It performs well across a range of problems.

Case 4: High crossover $p = 0.9$. Nearly

Differential Evolution: Parameter Analysis and Variants II

Roles of the Three Components

The differential weight w controls the exploration radius: set w too small and the algorithm cannot escape the current cluster of individuals; set w too large and the algorithm jumps around randomly. The anchor $\mathbf{a}$ determines which region of the space the perturbation is centered on: using a random individual (as in DE/rand) encourages diversity, while using the current best encourages faster convergence (DE/best). The crossover probability p controls how many dimensions of the current individual are replaced by the trial vector.

Particle Swarm Optimization: Inspiration and Update Equations I

Particle Swarm Optimization (PSO) was introduced by Kennedy and Eberhart in 1995, inspired by the collective movement of bird flocks and fish schools. The key observation is that animals in a group coordinate their movement using two types of information: their own personal experience (where they personally found food before) and the collective knowledge of the group (where the best food location known to anyone in the group is).

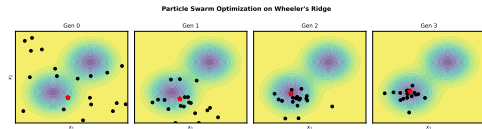
In PSO, each individual is called a *particle*. Unlike GA or DE individuals which only have a position, each particle also has a *velocity* that carries momentum from one iteration to the next.

PSO State per Particle

Each particle i maintains three quantities:

- $\mathbf{x}^{(i)}$: current position in the design space.
- $\mathbf{v}^{(i)}$: current velocity (direction and speed of movement).
- $\mathbf{x}_{\text{best}}^{(i)}$: the best position that particle i has personally ever visited (lowest f encountered so far).

In addition, the entire swarm shares one global variable: $\mathbf{x}^*$ ($\mathbf{x}$ star): the best position found by *any* particle at any



PSO applied to Wheeler's Ridge (a multi-modal benchmark function). The red star marks the global best position found so far. Particles (arrows showing velocity) converge toward two competing modes; the swarm eventually identifies the true global minimum.

Particle Swarm Optimization: Inspiration and Update Equations II

How to Read the Velocity Update

The velocity update formula has three additive terms. The first term $w\mathbf{v}$ is *inertia*: the particle tends to keep moving in the same direction it was already going. The second term $c_1 r_1 \odot (\mathbf{x}_{\text{best}} - \mathbf{x})$ is the *cognitive pull*: a force directed from the current position toward the particle's personal best — stronger when the particle is far from its personal best. The third term $c_2 r_2 \odot (\mathbf{x}^* - \mathbf{x})$ is the *social pull*: a force directed toward the global best position. The random vectors r_1 and r_2 ensure that these pulls are not deterministic, preventing particles from converging to a single point instantly. Typical values are $w = 0.1$, $c_1 = 0.25$, $c_2 = 2.0$, or alternatively $w = 0.9$, $c_1 = c_2 = 2.0$.

PSO: Python Implementation

```
1 import numpy as np
2
3 def particle_swarm(f, bounds, n_particles=30, k_max=100,
4                   w=0.1, c1=0.25, c2=2.0, rng=None):
5     rng = rng or np.random.default_rng()
6     lo, hi = np.array(bounds).T
7     n = len(lo)
8     # Initialise positions and velocities
9     pos = rng.uniform(lo, hi, (n_particles, n))
10    vel = rng.normal(0, 0.5, (n_particles, n)) # small initial velocities
11    x_best = pos.copy() # personal bests = initial positions
12    y_best = np.array([f(p) for p in pos]) # personal best objective values
13    g_best = x_best[np.argmin(y_best)].copy() # global best position
14
15    for _ in range(k_max):
16        r1 = rng.random((n_particles, n)) # random matrices for stochasticity
17        r2 = rng.random((n_particles, n))
18        # Velocity update: inertia + cognitive pull + social pull
19        vel = (w * vel
20              + c1 * r1 * (x_best - pos) # cognitive term
21              + c2 * r2 * (g_best - pos)) # social term
22        pos = np.clip(pos + vel, lo, hi) # move and enforce bounds
23        y = np.array([f(p) for p in pos])
24        # Update personal bests where improvement occurred
25        improved = y < y_best
26        x_best[improved] = pos[improved]
27        y_best[improved] = y[improved]
```

PSO: Intuition, Convergence, and Comparison with GA I

Three Forces Acting on Each Particle

Force 1: Inertia ($w\mathbf{v}$). Each particle carries momentum from its previous velocity. This prevents it from stopping abruptly when the cognitive and social forces cancel out, and it allows the particle to overshoot an attractor and explore beyond it. When w is large (close to 1), particles travel in long, sweeping arcs; when w is small (close to 0), particles change direction rapidly in response to the other two forces.

Force 2: Cognitive pull ($c_1 r_1(\mathbf{x}_{\text{best}} - \mathbf{x})$). This force pulls the particle back toward the best position it has personally visited. It acts as a form of individual memory: even if the particle has drifted away from a good region, this force brings it back.

Force 3: Social pull ($c_2 r_2(\mathbf{x}^* - \mathbf{x})$). This force pulls the particle toward the best position found by any particle in the entire swarm. It is the mechanism by

Computational Complexity

Per iteration: $O(m \cdot T_f + m \cdot n)$. Storage: $O(m \cdot n)$ for positions, velocities, and personal bests. Here m (m) is the number of particles, n (n) is the number of design variables, and T_f (T sub f) is the cost of one function evaluation.

PSO vs. Genetic Algorithms: Key Differences

Property	GA	PSO
Individual memory	no	yes (velocity + pbest)
Crossover	yes	no
Mutation	yes	inertia plays a similar role
Social info source	parent pairs	global best $\mathbf{x}^*$
Per-iter cost	$O(m(T_f + n))$	$O(m(T_f + n))$
Continuous $\mathbf{x}$	yes	yes (natural)
Discrete $\mathbf{x}$	yes (natural)	harder

Firefly Algorithm: Brightness-Based Attraction I

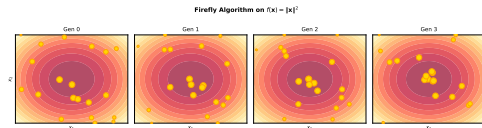
The Firefly Algorithm was proposed by Xin-She Yang in 2009. It is inspired by the bioluminescent signalling behaviour of fireflies: each firefly emits light, and less-bright fireflies are attracted toward brighter ones. In the optimization context, the *brightness* of an individual is inversely related to its objective value — a lower $f(\mathbf{x}^{(i)})$ means a brighter individual. Individuals with higher objective values (dimmer fireflies) move toward individuals with lower objective values (brighter fireflies).

Firefly Algorithm Update — Algorithm 9.8

At each iteration, for every ordered pair of individuals (i, j) such that $f(\mathbf{x}^{(j)}) < f(\mathbf{x}^{(i)})$ (meaning firefly j is brighter than firefly i):

- 1 Compute the Euclidean distance between the two individuals: $r = \|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\|$ where r (r) is a non-negative scalar measuring how far apart the two individuals are.
- 2 Compute the attraction of firefly j on firefly i :

$$\beta(r) = \beta_0 e^{-\gamma r^2}$$



The firefly algorithm minimising $f(\mathbf{x}) = \|\mathbf{x}\|^2$. Dot size encodes brightness (larger = brighter = lower f). Over iterations, all fireflies concentrate near the global minimum at the origin as dimmer individuals are attracted toward brighter ones.

Firefly Algorithm: Brightness-Based Attraction II

How to Read the Attraction Formula

The formula $\beta(r) = \beta_0 e^{-\gamma r^2}$ says: when two fireflies are close (r small), the attraction $\beta(r)$ is approximately β_0 — nearly maximal. As distance r grows, $\beta(r)$ decays exponentially fast due to the $e^{-\gamma r^2}$ factor (the negative exponent means the exponential shrinks as r grows). Physically: light absorption in the atmosphere reduces how strongly a distant firefly is perceived. The consequence for optimization: fireflies form local clusters around the best individuals, making the algorithm effective at exploiting multiple modes simultaneously.

Firefly Algorithm: Python Implementation

```
1 import numpy as np
2
3 def firefly(f, bounds, n=20, k_max=100,
4           alpha=0.3, beta0=1.0, gamma=1.0, rng=None):
5     rng = rng or np.random.default_rng()
6     lo, hi = np.array(bounds).T
7     dim = len(lo)
8     pop = rng.uniform(lo, hi, (n, dim))    # uniform initialisation
9
10    for _ in range(k_max):
11        y = np.array([f(p) for p in pop])
12        new_pop = pop.copy()
13        for i in range(n):
14            for j in range(n):
15                if y[j] < y[i]:                # j is brighter (lower objective)
16                    r = np.linalg.norm(pop[i] - pop[j])    # distance
17                    beta = beta0 * np.exp(-gamma * r**2)    # attraction
18                    # Move i toward j + random walk
19                    new_pop[i] = (new_pop[i]
20                                   + beta * (pop[j] - new_pop[i])
21                                   + alpha * rng.normal(0, 1, dim))
22        pop = np.clip(new_pop, lo, hi)
23
24    y = np.array([f(p) for p in pop])
25    return pop[np.argmin(y)]
```

*# Note: $O(n^2 * T_f)$ per iteration --- quadratic in population size*

Cuckoo Search: Levy Flights and Nest Abandonment I

Cuckoo Search was proposed by Xin-She Yang and Suash Deb in 2009. It is inspired by the brood-parasitism behaviour of cuckoo birds, which lay their eggs in the nests of other bird species. The analogy maps as follows: each cuckoo's egg is an optimization solution (a candidate design), and nests represent locations in the design space. A nest with a better egg (lower objective) survives to the next generation; a nest with a discovered foreign egg may be abandoned.

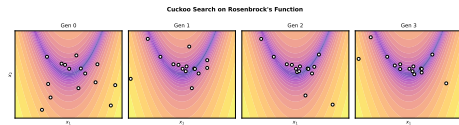
The key technical innovation of cuckoo search is its use of *Levy flights* for exploration. A Levy flight is a type of random walk in which the step lengths follow a heavy-tailed distribution (often approximated by a Cauchy distribution), enabling occasional very large jumps that help escape local minima.

Cuckoo Search Algorithm — Algorithm 9.9

The algorithm maintains a population of m (m) nests (candidate solutions), sorted by objective value.

- 1 **Levy flight step.** For each of m_s (m sub s) nests chosen in turn: pick a random nest index j , generate a new candidate via a Cauchy (heavy-tailed) flight:

$$\mathbf{x}' = \mathbf{x}^{(j)} + \text{Cauchy}(r) \cdot (\mathbf{u} - \mathbf{l})$$



Cuckoo search applied to Rosenbrock's function ($f(\mathbf{x}) = (1 - x_1)^2 + 100(x_2 - x_1^2)^2$, whose global minimum is at $(1, 1)$). The nests (white dots) gradually concentrate near $(1, 1)$ as the algorithm progresses.

Cuckoo Search: Levy Flights and Nest Abandonment II

Why Cauchy Flights Enable Better Global Exploration

A standard Gaussian random walk produces step sizes that are almost always small, because the tails of the Gaussian distribution decay extremely fast. The Cauchy distribution, by contrast, has *heavy tails*: large steps occur with much higher probability than under a Gaussian. This means that most iterations, the algorithm makes a small local refinement step — but occasionally it makes a large jump that can cross a valley or escape a basin of attraction. This mix of predominantly local steps with rare long-range jumps is mathematically described as a Levy flight pattern, and it is believed to be an efficient search strategy for complex, multi-modal landscapes.

Cuckoo Search: Python Implementation

```
1 import numpy as np
2
3 def cuckoo_search(f, bounds, pop_size=25, k_max=100,
4                   p_a=0.25, m_search_frac=1.0, r=0.01, rng=None):
5     rng = rng or np.random.default_rng()
6     lo, hi = np.array(bounds).T
7     n = len(lo)
8     m_search = max(1, int(m_search_frac * pop_size)) # nests that receive a step
9     m_abandon = max(1, int(p_a * pop_size))          # nests to abandon each iter
10
11     # Initialise: list of (x, y) pairs sorted by objective value
12     pop = [(x, f(x)) for x in rng.uniform(lo, hi, (pop_size, n))]
13     pop.sort(key=lambda t: t[1])
14
15     for _ in range(k_max):
16         # Step 1: Levy (Cauchy) flight for m_search nests
17         for i in range(m_search):
18             j = rng.integers(pop_size) # pick random nest
19             step = rng.standard_cauchy(n) * r * (hi - lo) # Cauchy step
20             x_new = np.clip(pop[j][0] + step, lo, hi)
21             y_new = f(x_new)
22             if y_new < pop[i][1]: # accept if better
23                 pop[i] = (x_new, y_new)
24         # Step 2: Sort and abandon m_abandon worst nests
25         pop.sort(key=lambda t: t[1])
26         for i in range(pop_size - m_abandon, pop_size):
27             x_new = rng.uniform(lo, hi, n) # random replacement
```

Cuckoo Search: Key Properties and Parameter Summary I

Levy and Cauchy Flights Explained

The Cauchy distribution, used here to approximate Levy flights, has the following properties relevant to optimization:

- Its probability density function has *heavier tails* than the Gaussian (normal) distribution. Where a Gaussian step of size 10σ is astronomically unlikely, a Cauchy step of the same size is merely uncommon.
- Most Cauchy-distributed steps are small (the peak of the distribution is at zero), providing local refinement in most iterations.
- Occasionally, a very large step occurs (“Levy flight”), which can jump the algorithm across a large basin to explore a completely different region.

Computational Complexity

Step	Cost per iteration
m_s Levy flights	$O(m_s \cdot T_f)$
Sort nests	$O(m \log m)$
Abandon and replace	$O(m_a \cdot T_f)$
Total per iter	$O(m \cdot T_f)$

The overall cost is linear in population size m — the same order as PSO and DE, and much cheaper than the firefly algorithm’s $O(m^2 T_f)$.

Parameter Summary and Typical Values

Parameter	Meaning	Typical value
m	population size	15–50
p_a	abandonment rate	0.25

9.7 Hybrid Methods: Combining Global and Local Search I

Population methods are excellent at *global exploration*: they maintain diversity across the design space and can locate the basin of attraction of the global minimum. However, they are often *slow at refining* a solution once they are near a minimum. Local search methods (gradient descent, Nelder-Mead, etc.) are the opposite: they converge quickly near a minimum but get trapped if they start far from the global optimum.

Hybrid methods combine both strategies: use a population method for global exploration and a local search method for local refinement. The key design question is: *should the local search move the individual to its local minimum, or should it only be used to compute a better objective value for selection purposes while the individual stays at its original position?*

Philosophy 1: Lamarckian Learning

Named after the French biologist Jean-Baptiste Lamarck, who proposed that organisms can pass on traits acquired during their lifetime. In the optimization context: run local search starting from each individual $\mathbf{x}^{(i)}$ and *physically move* the individual to the local minimum $\mathbf{x}_{\text{local}}^{(i)}$ that the local search finds. The individual's position in the population is permanently updated.

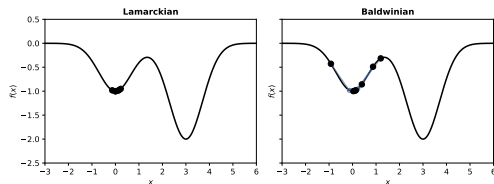


Illustration with $f(x) = -e^{-x^2} - 2e^{-(x-3)^2}$ (two local minima: a shallower one near $x = 0$ and the global minimum near $x = 3$).

Left (Lamarckian): all individuals near $x = 0$ move

9.7 Hybrid Methods: Combining Global and Local Search II

Key Takeaway: When to Use Each Philosophy

If the problem has few local minima and you need fast convergence, Lamarckian learning is effective. If the landscape is complex with many local minima and you are unsure whether the current population is near the global optimum, use Baldwinian learning to preserve the population's ability to explore multiple basins simultaneously. As a general rule of thumb: use Baldwinian when the global minimum may be far from where most individuals are currently located.

Baldwinian Hybrid GA: Python Example

```
1 import numpy as np
2 from scipy.optimize import minimize
3
4 def baldwinian_ga(f, bounds, pop_size=30, k_max=50,
5                  k_trunc=8, sigma=0.3, rng=None):
6     rng = rng or np.random.default_rng()
7     lo, hi = np.array(bounds).T
8     n = len(lo)
9     pop = rng.uniform(lo, hi, (pop_size, n))
10
11     for _ in range(k_max):
12         # Baldwinian: use local-search value for selection, keep original position
13         def local_val(x):
14             res = minimize(f, x, method='Nelder-Mead',
15                           options={'maxiter': 20, 'xatol': 1e-4})
16             return res.fun # return improved objective, NOT the improved position
17
18         y = np.array([local_val(x) for x in pop]) # pop positions unchanged
19
20         # Standard GA: truncation selection + single-point crossover + Gaussian mutation
21         parents = pop[np.argsort(y)[:k_trunc]]
22         new_pop = []
23         while len(new_pop) < pop_size:
24             a, b = parents[rng.choice(k_trunc, 2, replace=False)]
25             cp = rng.integers(1, n)
26             child = np.concatenate([a[:cp], b[cp:]])
27             child += rng.normal(0, sigma, n) * (rng.random(n) < 1/n)
```

Lamarckian vs. Baldwinian: Worked Example and Comparison I

Worked Example 9.2 (from textbook)

Consider the one-dimensional function:

$$f(x) = -e^{-x^2} - 2e^{-(x-3)^2}$$

This function has two local minima: a shallower one near $x = 0$ (where $f(0) \approx -1$) and the global minimum near $x = 3$ (where $f(3) \approx -2$). Suppose the initial population is clustered near $x = 0$.

Under **Lamarckian** learning: each individual is moved to its nearby local minimum at $x \approx 0$. All individuals now sit at the local minimum, and all future crossover and mutation produces offspring also near $x = 0$. The probability of discovering the global minimum at $x \approx 3$ becomes vanishingly small.

Under **Baldwinian** learning: the selection step uses the local-search-improved values (which are all

Comparison Table

Property	Lamarckian	Baldwinian
Exploits local info	Strongly	Weakly
Preserves diversity	No	Yes
Convergence speed	Fast locally	Moderate
Global reach	Low (premature)	Higher
Extra cost	$O(T_{\text{local}})$	$O(T_{\text{local}})$
Risk of getting stuck	High	Low

Both methods incur the same additional cost: $O(T_{\text{local}})$ per individual per generation, where T_{local} (T sub local) is the cost of running the local optimizer. Since Nelder-Mead or a few gradient steps are typically much cheaper than solving the full problem, this overhead is usually acceptable.

Recommendation

Chapter 9 Summary I

This chapter introduced the family of *population methods* for optimization — algorithms that maintain a collection of m (m) candidate solutions and evolve them collectively over many generations. Unlike single-point methods, population methods share information across the design space and are naturally suited to avoiding local minima.

Genetic Algorithms (Section 9.2)

GAs are biologically inspired, applying selection, crossover, and mutation each generation. Selection (truncation, tournament, or roulette-wheel) rewards lower-objective individuals. Crossover (single-point, two-point, uniform, or interpolation) combines parent chromosomes. Mutation introduces random perturbations to prevent premature convergence and enable exploration of new gene values. GAs work naturally on both discrete and continuous design spaces.

Firefly Algorithm (Section 9.5)

The firefly algorithm models pairwise attraction based on brightness (fitness): dimmer individuals move toward brighter ones with an attraction strength that decays exponentially with distance. The attraction formula $\beta(r) = \beta_0 e^{-\gamma r^2}$ creates natural clustering around good solutions. The main drawback is $O(m^2)$ cost per iteration due to the all-pairs comparison.

Cuckoo Search (Section 9.6)

Cuckoo search uses heavy-tailed (Cauchy/Levy)

Differential Evolution (Section 9.3)

Method Comparison: Complexity and Practical Guidance I

Algorithm Complexity and Parameter Summary

Method	Per-iteration cost	Key parameters	Strengths	Best for
Genetic Algorithm	$O(m(T_f + n \log m))$	m, k, λ, σ	Flexible encoding	Discrete + continuous
Differential Evolution	$O(m \cdot T_f)$	m, p, w	Self-adapting step size	Continuous, multi-modal
PSO	$O(m \cdot T_f)$	m, w, c_1, c_2	Fast, simple	Continuous, fast conv.
Firefly	$O(m^2 T_f)$	$m, \alpha, \beta_0, \gamma$	Pairwise attraction	Multi-modal, small m
Cuckoo Search	$O(m \cdot T_f)$	m, p_a, r	Long-range jumps	Rugged, continuous

Method Comparison: Complexity and Practical Guidance II

Practical Guidance: Which Method to Use?

Unknown landscape, first attempt. Start with PSO or DE. Both have few parameters, are fast (linear cost in m), and are robust across a wide range of continuous problems. PSO is particularly easy to tune (just adjust w and c_2).

Mixed-integer or combinatorial problem. Use a GA with an appropriate crossover and mutation scheme for the encoding (bit-flip mutation for binary genes, Gaussian for continuous genes).

Highly multi-modal landscape. Cuckoo search (Levy flights enable long-range jumps between basins) or the firefly algorithm (pairwise attraction can maintain multiple sub-populations around different modes simultaneously).

Need fast local refinement. Wrap any population method with Baldwinian (preferred) or Lamarckian local search.

Limited evaluation budget. Population size m is the primary lever: a smaller m uses fewer evaluations per generation but may miss the global optimum. Increasing $k_{\max}$ while decreasing m (or vice versa) trades breadth for depth.

Selected Exercises with Worked Solutions I

9.1 *What is the purpose of the selection operation in genetic algorithms?*

Solution. Selection serves to bias reproductive success toward individuals with lower objective values (higher fitness), ensuring that the gene pool of the next generation is dominated by better-performing individuals. Without selection, the GA would simply perform random crossover and mutation with no pressure toward improvement — it would behave like a random search. Selection is the mechanism by which useful genetic “building blocks” (combinations of gene values that contribute to good objective values) are preserved and amplified across generations.

9.2 *Why does mutation play a fundamental role in GAs? If you suspect there is a better optimum elsewhere, how should you adjust λ (lambda)?*

Solution. Mutation is fundamental because crossover alone can only combine gene values that already exist in the population. If the optimal solution requires a gene value that was not present in any individual at initialisation, it can *never* be discovered by crossover. Mutation introduces new gene values by random perturbation, enabling the GA to explore regions of the design space that were not represented in the initial population. If a better optimum is suspected elsewhere (for example, because the current best has not improved for many generations), *increase* λ above the default $1/m$ to generate more exploration per generation.

Selected Exercises with Worked Solutions II

9.3 *PSO converges to a non-global minimum. How can you adjust the parameters?*

Solution. Premature convergence in PSO usually means the social force ($c_2 r_2 (\mathbf{x}^* - \mathbf{x})$) is overwhelmingly pulling all particles toward the current global best, which happens to be a local (not global) minimum. To address this: (1) increase the number of particles $n_{\text{particles}}$ so the initial population is more spread out and more likely to have at least one particle near the global minimum; (2) increase the cognitive coefficient c_1 relative to c_2 , strengthening each particle's attraction to its own personal best and weakening the social force; (3) increase the inertia weight w to give particles more momentum to escape the attractor basin of the local minimum.

9.4 *Compare five DE variants on Ackley's function and explain the results.*

Solution. (1) $w = 0$: worst performance — the trial vector $\mathbf{z} = \mathbf{a} + 0 \cdot (\mathbf{b} - \mathbf{c}) = \mathbf{a}$ contains only existing population values, so no new gene values outside the current range can be generated. (2) $\mathbf{a} = \mathbf{x}^{(i)}$: biased toward local search around each individual's own position, making it hard to escape local minima. (3) Standard $p = 0.5, w = 0.5$: balanced — a good default. (4) $p = 0.9$: nearly always replaces with the trial vector, performing almost as well as full replacement. (5) $p = 1.0$ (guaranteed crossover): best overall — every dimension of every individual is updated at every iteration, maximising information flow from the differential perturbation.

References I

- M. J. Kochenderfer & T. A. Wheeler, *Algorithms for Optimization*, 2nd ed., MIT Press, 2026. Chapter 9.
- J. H. Holland, *Adaptation in Natural and Artificial Systems*, MIT Press, 1992. (Original treatment of Genetic Algorithms and the schema theorem.)
- R. Storn & K. Price, “Differential Evolution — a simple and efficient heuristic for global optimization over continuous spaces,” *Journal of Global Optimization*, 11(4):341–359, 1997.
- J. Kennedy & R. Eberhart, “Particle Swarm Optimization,” *Proceedings of IEEE International Conference on Neural Networks (ICNN)*, pp. 1942–1948, 1995.
- X.-S. Yang, “Firefly Algorithms for Multimodal Optimization,” in *Stochastic Algorithms: Foundations and Applications (SAGA)*, Lecture Notes in Computer Science, vol. 5792, Springer, 2009.
- X.-S. Yang & S. Deb, “Cuckoo Search via Levy Flights,” in *Proceedings of the World Congress on Nature & Biologically Inspired Computing (NaBIC)*, pp. 210–214, IEEE Press, 2009.